Gradient Descent

Abbiamo visto che l'ERM trova un predittore h_S che è un buon approssimatore di h^*_H ma non sappiamo come calcolarlo in modo efficiente

Sia H rappresentabile dal vettore $\theta \in \mathbb{R}^d$

Il nostro obiettivo è ora minimizzare $\tilde{R}_S(h_\theta)$ in $\mathbb{R}^d$, che si traduce nel trovare quel θ t.c.

$$\nabla \tilde{R}_S(h_\theta) = 0$$

Ass

In realtà la sola minimizzazione non basta in quanto si potrebbe incappare in un minimo locale o in un punto di sella

Passi base per la discesa del gradiente

Per semplicità chiamiamo $\tilde{R}_S(h_\theta) = \tilde{J}(\theta)$

La discesa del gradiente si basa sull'eseguire iterativamente la formula

$$\theta^{(k+1)} = \theta^{(k)} - \eta \nabla \tilde{J}(\theta^{(k)}) \quad \text{con } k = 0, 1, \dots \text{ numero di epoche}$$

Vedendolo componente per componente abbiamo

$$\theta_i^{(k+1)} = \theta_i^{(k)} - \eta \frac{\partial \tilde{J}(\theta^{(k)})}{\partial \theta_i}$$

Questa formula per esser trattata meglio viene divisa in:

a) Gradient Signal:

$$g_{i,k} = \frac{\partial \tilde{J}(\theta^{(k)})}{\partial \theta_i}$$ → Dice la direzione in cui andare

b) Learning Rate

$$\eta_{i,k} = \eta$$ → Quanto a lungo seguiamo il gradiente

c) Incremento

$$\Delta_{i,k} = -\eta_{i,k} g_{i,k}$$ → Ci dice di quanto ci stiamo muovendo

d) Update:

$$\theta_j^{(k+1)} = \theta_j^{(k)} + \Delta_{j,k}$$ ↝ Come calcoliamo il nuovo vettore

Definito il Modus operandi del Gradient Descent vengono ora mostrati dei metodi per il calcolo di $g_{j,k}$, $\Delta_{j,k}$ e $\tilde{r}$

Metodi per la stima del gradient Signal

Il gradient signal viene stimato secondo i seguenti 3 metodi:

- Batch gradient Descent
- Stochastic gradient Descent
- Mini-Batch gradient Descent

Vediamoli più nello specifico

Batch Gradient Descent

Secondo il Batch Gradient descent il $\tilde{v}(\theta)$ calcolato ad ogni epoca è l'intero $\widetilde{RL}_{\tilde{c}}(h_\theta)$

Infatti:

$$g_{j,k} = \frac{1}{|\tilde{C}|} \sum_{(x,\delta)\in\tilde{C}} \frac{\partial L(h_\theta(x), \delta)}{\partial \theta_j}\Big|_{\theta^{(k)}}$$

Utilizzando questo metodo si ha che $\tilde{v}$ converge ad un minimo globale se $\tilde{v}$ è convessa oppure ad un minimo locale altrimenti

Il problema principale però è che andando a calcolare $\widetilde{RL}_{\tilde{c}}$ ad ogni epoca risulta molto costoso quando $\tilde{C}$ è grande

Stochastic gradient Descent

SGD

Per diminuire la complessità computazionale del Batch Gradient Descent si è progettato lo Stochastic Gradient Descent

Ad ogni epoca calcoliamo $g_{j,k}$ come:

- Si seleziona $(x,\delta)\in\tilde{C}$ unif. a random
- Calcoliamo $g_{j,k}$ come

$$g_{j,k} = \frac{\partial L(h_\theta(x), \delta)}{\partial \theta_j}\Big|_{\theta^{(k)}}$$

Vediamo come essendo $(x,\delta)\in_u \tilde{C}$ il valor medio di SGD è il Batch Gradient Descent ma presenta una Varianza molto elevata

Infatti nonostante il costo computazionale sia diminuito di molto, questo non converge mai veramente ad un minimo a meno che non si cambia η in base all'epoca corrente

↳ Ci girerà sotto intorno

Una condizione sufficiente è fare in modo che

$$\sum_{k} \eta_k = \infty \quad e \quad \sum_{k} \eta_k^2 < \infty$$

↳ Verrà approfondito in seguito

Mini Batch Gradient Descent

Un buon compromesso tra l'accuratezza del Batch e l'efficienza del SGD è il Mini-Batch

Fissato m t.c. $1 \leq m \leq |\mathcal{C}|$ Ad ogni epoca calcoliamo $g_{j,k}$ come:

- Selezioniamo un'insieme $B \subset \mathcal{C}$ u.a.r. t.c. $|B| = m$
- Calcoliamo $g_{j,k}$ come:

$$g_{j,k} = \frac{1}{m} \sum_{(x,\theta) \in B} \frac{\partial L(h_\theta(x), \theta)}{\partial \theta_j} \Big|_{\theta^{(k)}}$$

In questo modo stiamo diminuendo la varianza riscontrata in SGD di un fattore m senza andare a pagare il costo elevato del Batch

>>>

Possiamo notare come il Mini-Batch sia una generalizzazione sia del SGD e del Batch in quanto per $m = 1$ abbiamo l'SGD mentre per $m = |\mathcal{C}|$ abbiamo il Batch

Metodi per la gestione dell'incremento

Vediamo ora due metodi per la modulazione dell'incremento $\Delta_{j,k}$:

- Momentum
- Nesterov Accellerated Gradient

Se il nuovo passo della discesa segue la stessa direzione del passo precedente allora aumento di un po' l'incremento

Momentum

La logica dietro al Momentum si basa su un'analogia fisica

Infatti se vediamo θ come una massa soggetta alla forza $F = -\eta \nabla \tilde{V}(\theta)$ viene naturale pensare che ad ogni passo questa mantenga una velocità dipesa da tutti i movimenti passati

Abbiamo quindi

$$\theta_j^{(k+1)} = \theta_j^{(k)} + V_j^{(k+1)} \quad \text{dove} \quad V_j^{(k+1)} = \gamma V_j^{(k)} - \eta g_{j,k} \quad e \quad \gamma \in (0,1)$$

Srotolando la ricorsione di $v_j^{(k+1)}$ ponendo $v_j^{(0)} = 0$ abbiamo

$$v_j^{(k+1)} = -\eta \sum_{r=0}^{k} \gamma^{k-r} g_{j,r}$$

Da cui si nota l'importanza di γ, infatti più γ è vicino ad 1 più la velocità viene conto dei gradienti passati

Più è vicino a 0 più dimentica il passato

Con $\gamma = 0$ abbiamo l'incremento classico senza Momentum

Il vantaggio sostanziale nell'utilizzo del momentum è che

- Se la direzione del gradiente al passo $k+1$ è la stessa del passo k allora l'incremento sarà maggiore
- Se la direzione al passo $k+1$ è diversa da quella del passo k allora l'incremento diminuirà

Nesterov Accelerated Gradient

Il Nesterov Accelerated Gradient basandosi sulla logica del Momentum va a modificare il punto in cui viene calcolato il gradiente

Chiamiamo $\tilde{\theta}^{(k)}$ una posizione di look-ahead:

$$\tilde{\theta}^{(k)} = \theta^{(k)} + \gamma v^{(k)}$$

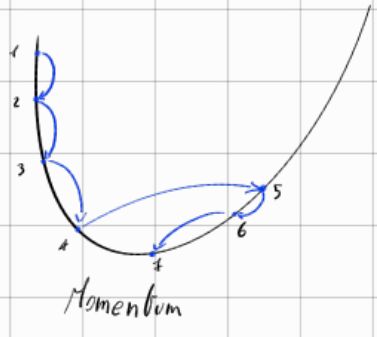
Da dove poi andiamo a calcolare il gradiente da cui dipende la velocità

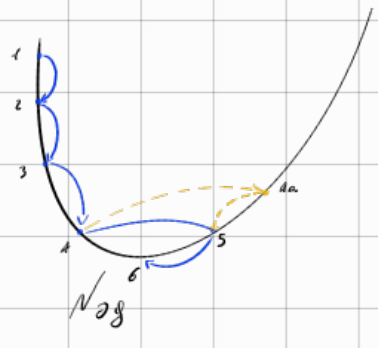
$$v_j^{(k+1)} = \gamma v_j^{(k)} - \eta \tilde{g}_{j,k} \rightsquigarrow \text{Gradiente calcolato in } \tilde{\theta}$$

Che poi aggiungiamo al punto $\theta^{(k)}$

$$\theta^{(k+1)} = \theta^{(k)} + v^{(k+1)}$$

Si sta modificando il Momentum in modo tale che l'eventuale overshoot dato dalla velocità venga mitigato in presenza di un cambio di direzione





Adaptive Learning Rates

Abbiamo visto come gestire sia l'incremento che il gradient signal senza mai andare a parlare del learning rate η

Fino ad ora η è rimasto costante in relazione alle features, questo però può portare a diverse complicazioni

Infatti bisognava bilanciare η in modo tale che non fosse troppo basso per features poco frequenti né troppo alto per il resto

Per questo introduciamo ora gli Adaptive Method che permettono di variare $\eta_{i,k}$ in base ai gradienti ricevuti da i nel passato

Adagrad

Adagrad gestisce il Learning rate in modo che questo decresca in relazione alla grandezza del gradient signal

Sia $G_{\tilde{v},k} = \sum_{i=0}^{k} g^2_{\tilde{v},i}$

Calcoliamo

$$\eta_{\tilde{v},k} = \frac{\eta}{\sqrt{G_{\tilde{v},k} + \varepsilon}}$$

L'unica limitazione di Adagrad è che $G_{\tilde{v},k}$ può solo che crescere durante le varie epoche

Infatti per Training lunghi il rate diventa troppo piccolo per aver degli incrementi sostanziali

RMSprop

exponential moving average

RMSprop risolve la criticità di Adagrad sostituendo la somma $G_{\tilde{v},k}$ con un'EMA dei quadrati dei gradienti

Definiamo l'EMA come

$$\tilde{G}_{\tilde{v},k} = \gamma \tilde{G}_{\tilde{v},k-1} + (1-\gamma) g^2_{\tilde{v},k} \quad \text{con } G_{\tilde{v},0} = 0 \text{ e } \gamma = 0.9$$

di un fattore $(1-\gamma)\gamma^{k-i}$

In modo che all'avanzare di k $g_{\tilde{v},i}$ con $i<k$ vada a pesare sempre meno all'interno di $\tilde{G}_{\tilde{v},k}$

Andiamo a calcolare quindi

$$\eta_{\tilde{v},k} = \frac{\eta}{\sqrt{\tilde{G}_{\tilde{v},k} + \varepsilon}}$$

Resta sempre il problema di settare un $\eta = 10^{-3}$ su cui calcolare poi $\eta_{\tilde{v},k}$

Adadelta

Adadelta rimuove questa dipendenza da η sostituendolo con un' EMA sull'incremento $\Delta_{\tilde{v},k}$

Definendo quindi

$$\overline{G}_{\tilde{v},k} = \gamma \overline{G}_{\tilde{v},k-1} + (1-\gamma)\Delta_{\tilde{v},k}$$

Abbiamo quindi

$$\eta_{\tilde{v},k} = \frac{\sqrt{\overline{G}_{\tilde{v},k-1} + \varepsilon}}{\sqrt{\widehat{G}_{\tilde{v},k} + \varepsilon}}$$

→ Si usa $\overline{G}_{\tilde{v},k-1}$ invece di $\overline{G}_{\tilde{v},k}$ visto che $\Delta_{\tilde{v},k}$ deve ancora essere calcolato

Adam

Adam, invece di gestire solo l'evoluzione del rate, va a lavorare sull'incremento modificando il gradient signal

In Sobbi andiamo ora a definire due EMA:

$$\widetilde{H}_{\tilde{v},k} = \beta_1 \widetilde{H}_{\tilde{v},k-1} + (1-\beta_1) g_{\tilde{v},k} \quad \text{con } \beta_1 \simeq 0{,}9$$

→ Stiamo applicando un EMA che generalizza il Momentum in modo da applicarlo ad un Metodo Adattivo

$$\widetilde{G}_{\tilde{v},k} = \beta_2 \widetilde{G}_{\tilde{v},k-1} + (1-\beta_2) g^2_{\tilde{v},k} \quad \text{con } \beta_2 \simeq 0{,}999$$

$$\widetilde{G}_{\tilde{v},0} = (1-\beta_2) g^2_{\tilde{v},0} = (1-0{,}999) g^2_{\tilde{v},0} = \frac{g^2_{\tilde{v},0}}{1000}$$

Notiamo però che per i primi passi $\widetilde{G}_{\tilde{v},k}$ e $\widetilde{H}_{\tilde{v},k}$ sono molto più piccoli rispetto a $g_{k,\tilde{v}}$

Per risolvere questa problematica si va ad eliminare il bias applicando

$$\widehat{H}_{\tilde{v},k} = \frac{\widetilde{H}_{\tilde{v},k}}{1-\beta_1^{k-1}} \quad \text{e} \quad \widehat{G}_{\tilde{v},k} = \frac{\widetilde{G}_{\tilde{v},k}}{1-\beta_2^{k-1}}$$

In questo modo per k piccoli i valori di $g_{\tilde{v},k}$ non viene disturbato troppo dall'EMA e per k grandi sia $1-\beta_1^{k-1}$ che $1-\beta_2^{k-1}$ tendono ad 1 lasciando invariato l'EMA

Detto questo andiamo quindi a calcolare sia l'incremento che il Learning rate

$$\eta_{\tilde{v},k} = \frac{\eta}{\sqrt{\widehat{G}_{\tilde{v},k} + \varepsilon}} \quad \text{e} \quad \Delta_{\tilde{v},k} = -\eta_{\tilde{v},k} \widehat{H}_{\tilde{v},k}$$

→ Come in RMSprop

→ A quanto pare Adam fa in modo che ci si sposti sempre di un fattore η mentre con la discesa classica questo si incrementa anche in relazione al gradient signal

In questo modo si ottiene che quando il segno del gradiente rimane consistente il rapporto $\frac{\widehat{H}}{\sqrt{\widehat{G}}}$ si avvicina ad 1

Per cui ogni parametro si muove di un fattore η

Grazie a questa caratteristica Adam è lo standard per il Deep Learning

Weight Decay

Spesso si ha a che fare con Dataset con un'alta dimensionalità (→ di molto grandi)

Questo porta solitamente ad una complessità eccessiva

Per questo è stato introdotto il Weight Decay in modo tale da: (→ Limitando così anche l'overfitting)

Diminuire la complessità del Training ponendo un freno ai parametri θ_j

Questo freno si ottiene aggiungendo il termine $\frac{\lambda}{2}\|\theta\|^2$ alla Loss in modo che (→ Lambda verso approfondito Sta poco)

$$\tilde{J}(\theta) = \tilde{R}_{\tilde{z}}(h_\theta) + \frac{\lambda}{2}\|\theta\|^2$$

Per cui avremo poi un'incremento della forma

$$\theta_j^{(k+1)} = \theta_j^{(k)} - \eta\left(\frac{\partial \tilde{R}_{\tilde{z}}(h_\theta)}{\partial \theta_j} + \lambda\theta_j^{(k)}\right) = (1-\lambda\eta)\theta_j^{(k)} - \eta\frac{\partial \tilde{R}_{\tilde{z}}(h_\theta)}{\partial \theta_j}$$

Con questo incremento noi stiamo ottenendo due risultati:

i) Minimizziamo la il rischio empirico ⇝ Dato dal fattore $-\eta\frac{\partial \tilde{R}_{\tilde{z}}(h_\theta)}{\partial \theta_j}$

ii) Evitiamo di avere parametri eccessivamente grandi ⇝ Grazie al fattore $(1-\lambda\eta)\theta_j^{(k)}$

Per quanto riguarda λ è il parametro che gestisce lo smorzamento

i) Più λ è grande più portiamo i parametri verso lo 0

ii) Più λ è piccolo meno smorziamo θ

↳ Per $\lambda = 0$ abbiamo la classica discesa del gradiente

AdamW

Il weight decay così come è applicato ad Adam ne va a "sporcare" i risultati

Per questo è stato ideato AdamW che va a separare le due operazioni:

i) Si cerca di minimizzare la Loss mediante il classico Adam

ii) Solo dopo viene applicato il Weight decay